UNIT_4_PQ_QB_VHA

215 For x = np.array([5, 15, 25, 35, 45, 55]) and y = np.array([5, 20, 14, 32, 22, 38]), apply simple linear regression using scikit learn library and calculate calculate R squared, coeficient and intercept. Predict the y values for x = np.arange(5). (Don't split data for training/testing)

In [1]:
```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
x = np.array([5, 15, 25, 35, 45, 55]).reshape(-1, 1)
y = np.array([5, 20, 14, 32, 22, 38])
# Create and fit the model
model = LinearRegression()
model.fit(x, y)
# Calculate R-squared, coefficient, and intercept
r_squared = model.score(x, y)
coefficient = model.coef_[0]
intercept = model.intercept_
# Predict y values for x = np.arange(5)
x_pred = np.arange(5).reshape(-1, 1)
y_pred = model.predict(x_pred)
# Print the results
print(f"R-squared: {r_squared}")
print(f"Coefficient: {coefficient}")
print(f"Intercept: {intercept}")
print(f"Predicted y values for x = np.arange(5): {y_pred}")
```

```
R-squared: 0.7158756137479542
Coefficient: 0.54
Intercept: 5.633333333333329
Predicted y values for x = np.arange(5): [5.63333333 6.17333333 6.71333333 7.253333
33 7.79333333]
```

216 Given a dataset with 'SAT' scores as independent variables and 'GPA' as the dependent variable, calculate R squared, coeficient and intercept using linear regression and scikitlearn library. (Don't split data for training/testing)

In [6]:
```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Load the dataset
df = pd.read_csv('SATGPA.csv')

# Independent variable (SAT scores) and dependent variable (GPA)
X = df[['SAT']]
y = df['GPA']

# Create and fit the model
model = LinearRegression()
model.fit(X, y)

# Calculate R-squared, coefficient, and intercept
r_squared = model.score(X, y)
coefficient = model.coef_[0]
intercept = model.intercept_
```

```
# Print the results
print(f"R-squared: {r_squared}")
print(f"Coefficient: {coefficient}")
print(f"Intercept: {intercept}")
```

```
R-squared: 0.40600391479679754
Coefficient: 0.001655688050092815
Intercept: 0.2750402996602781
```

217 Given a real estate price size year dataset, implement multiple linear regression using scikitlearn library. Using the model, make a prediction about an apartment price with size 750 sq.ft. for 2009.Also Calculate R squared, coeficient and intercept. (Don't split data for training/testing)

In [9]:
```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
# Load the dataset
df = pd.read_csv('real_estate.csv')

# Independent variables (size and year) and dependent variable (price)
x = df[['size', 'year']]
y = df['price']

# Create and fit the model
model = LinearRegression()
model.fit(x, y)

# Calculate R-squared, coefficients, and intercept
r_squared = model.score(x, y)
coefficients = model.coef_
intercept = model.intercept_

# Make a prediction for an apartment with size 750 sq.ft. for the year 2009
prediction = model.predict([[750, 2009]])

# Print the results
print(f"R-squared: {r_squared}")
print(f"Coefficients: {coefficients}")
print(f"Intercept: {intercept}")
print(f"Predicted price for a 750 sq.ft. apartment in 2009: {prediction[0]}")
```

```
R-squared: 0.7764803683276793
Coefficients: [ 227.70085401 2916.78532684]
Intercept: -5772267.01746328
Predicted price for a 750 sq.ft. apartment in 2009: 258330.34465994593
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\base.py:439: UserWarning: X doe
s not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
```

218 Predict salary based on job position of 6.5 using polynomial regression with a degree of 3 and scikit learn library for the given 'Position_Salaries.csv' dataset. (Don't split data for training/testing)

In [11]:
```
import pandas as pd
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Step 1: Load the dataset
data = pd.read_csv('Position_Salaries.csv')
```

```
# Step 2: Extract features and target variable
X = data.iloc[:, 1:2].values  # Independent variable (job position)
y = data.iloc[:, 2].values    # Dependent variable (salary)

# Step 3: Fit polynomial regression model
poly_reg = PolynomialFeatures(degree=3)
X_poly = poly_reg.fit_transform(X)

lin_reg = LinearRegression()
lin_reg.fit(X_poly, y)

# Step 4: Make predictions
job_position = [[6.5]]  # Job position for which we want to predict salary
predicted_salary = lin_reg.predict(poly_reg.transform(job_position))

print("Predicted salary for job position 6.5:", predicted_salary[0])
```

```
Predicted salary for job position 6.5: 133259.46969697624
```

219 For x = np.arange(0, 30) and y = np.array([3, 4, 5, 7, 10, 8, 9, 10, 10, 23, 27, 44, 50, 63, 67, 60, 62, 70, 75, 88, 81, 87, 95, 100, 108, 135, 151, 160, 169, 179]), apply polynomial regression using scikit learn library and calculate R squared, coeficient and intercept. Predict the y values for x = np.arange(5). (Don't split data for training/testing)

In [13]:
```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Step 1: Prepare the data
x = np.arange(0, 30)
y = np.array([3, 4, 5, 7, 10, 8, 9, 10, 10, 23, 27, 44, 50, 63, 67, 60, 62, 70, 75

# Step 2: Fit polynomial regression model
poly_reg = PolynomialFeatures(degree=3)
X_poly = poly_reg.fit_transform(x.reshape(-1, 1))

lin_reg = LinearRegression()
lin_reg.fit(X_poly, y)

# Step 3: Calculate R squared, coefficient, and intercept
r_squared = lin_reg.score(X_poly, y)
coefficient = lin_reg.coef_
intercept = lin_reg.intercept_

print("R squared:", r_squared)
print("Coefficient:", coefficient)
print("Intercept:", intercept)

# Step 4: Make predictions for x = np.arange(5)
new_x = np.arange(5)
X_new_poly = poly_reg.transform(new_x.reshape(-1, 1))
predicted_y = lin_reg.predict(X_new_poly)

print("Predicted y values for x = np.arange(5):", predicted_y)
```

```
R squared: 0.9760588291456355
Coefficient: [ 0.          3.38954229 -0.03098624  0.00447358]
Intercept: -3.1958699902266545
Predicted y values for x = np.arange(5): [-3.19586999  0.16715964  3.49505824  6.81
466728 10.15282821]
```

220 "Write a program to make a model based on linear regression for the following dataframe created from a csv file named “Package.csv” of x and y which follows equation y = a + bx. Write a program which can predict value of y based on any value of x, also write code to find value of a and b in above equation.

In [17]:
```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Step 1: Load the dataset
data = pd.read_csv("Placement.csv")

# Step 2: Extract features and target variable
X = data['cgpa'].values.reshape(-1, 1)  # Independent variable (x)
y = data['package'].values                  # Dependent variable (y)

# Step 3: Fit linear regression model
lin_reg = LinearRegression()
lin_reg.fit(X, y)

# Step 4: Make predictions for any given value of x
def predict_y(x_value):
    return lin_reg.predict([[x_value]])

# Step 5: Find coefficients
intercept = lin_reg.intercept_
coefficient = lin_reg.coef_[0]

print("Intercept (a):", intercept)
print("Coefficient (b):", coefficient)

# Example usage:
# Predict y for a given x value
x_value = 5
predicted_y = predict_y(x_value)
print("Predicted y for x =", x_value, ":", predicted_y[0])
```

```
Intercept (a): -0.9856779462557332
Coefficient (b): 0.5695912947937534
Predicted y for x = 5 : 1.8622785277130336
```

221 "Write a program to make a model based on linear regression for the following dataframe created from a csv file named “data.csv” of x1 and y which follows equation y = a+bx1. Write a program which can predict value of y based on any value of x, also write code to find value of a and b in above equation. Given Data in csv file:"

In [20]:
```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
d={"x1":[60,62,67,70,71,72,75,78],
"y":[140,155,159,179,192,200,212,215]}
df=pd.DataFrame(d)
x=df[["x1"]]
y=df["y"]
```

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=2)
from sklearn.linear_model import LinearRegression
lr=LinearRegression()
lr.fit(x_train,y_train)
y_pred=lr.predict(x_test)
m=lr.coef_
print(m)
c=lr.intercept_
print(c)
from sklearn.metrics import mean_squared_error
print(mean_squared_error(y_test, y_pred))
```

```
[4.64238411]
-142.34768211920536
56.54020656988712
```

222 "Write a program to create a Model using linear regression to predict the price of house using the csv file provided named “Housing.csv”. Do the required process in the data before making a model. Find predicted values, co-efficients, intercept and mean squared error.

In [27]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
df=pd.read_csv("HousingData.csv")
df=df.dropna()
# Linear Regression
# Let's use multiple features to predict the housing prices (MEDV)
X = df.drop('MEDV', axis=1)
y = df['MEDV']
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_st
# Create and train the linear regression model
model = LinearRegression()
model.fit(X_train, y_train)
# Make predictions
y_pred = model.predict(X_test)
# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("Mean Squared Error:", mse)
print("R-squared:", r2)
print(model.coef_)
print(model.intercept_)
```

```
Mean Squared Error: 31.45404766495098
R-squared: 0.6270849941673178
[-1.12187394e-01  4.24404148e-02  2.56728238e-02  1.98383708e+00
 -1.70792571e+01  4.25809072e+00 -2.17413906e-02 -1.42418883e+00
  2.35587949e-01 -1.19971379e-02 -9.75834850e-01  9.59377961e-03
 -3.88619588e-01]
33.65240504056575
```

223 "Write a program to create a Model using linear regression to predict the student scores using the csv file provided named “student_scores.csv”.

Do the required process in the data before making a model. Find predicted values, co-efficients, intercept and mean squared error.

In [3]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Step 1: Load the dataset
data = pd.read_csv("student_scores.csv")

# Step 2: Data preprocessing
# Check for any missing values
missing_values = data.isnull().sum()
print("Missing values:\n", missing_values)

# Step 3: Split the dataset into features and target variable
X = data[['Hours']]  # Features (independent variable: hours studied)
y = data['Scores']   # Target variable (scores obtained)

# Step 4: Fit linear regression model
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_st

lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

# Step 5: Make predictions
predicted_scores = lin_reg.predict(X_test)

# Step 6: Calculate coefficients and intercept
coefficients = lin_reg.coef_
intercept = lin_reg.intercept_

print("Coefficients:", coefficients)
print("Intercept:", intercept)

# Step 7: Calculate mean squared error
mse = mean_squared_error(y_test, predicted_scores)
print("Mean Squared Error:", mse)
```

```
Missing values:
 Hours     0
Scores    0
dtype: int64
Coefficients: [9.68207815]
Intercept: 2.826892353899737
Mean Squared Error: 18.943211722315272
```

224 "Write a program to create a Model using linear regression to predict the gas consumption using the csv file provided named “petrol_consumption.csv”. Do the required process in the data before making a model. Find predicted values, co-efficients, intercept and mean squared error.

In [4]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Step 1: Load the dataset
data = pd.read_csv("petrol_consumption.csv")
```

```
# Step 2: Data preprocessing
# Check for any missing values
missing_values = data.isnull().sum()
print("Missing values:\n", missing_values)

# Step 3: Split the dataset into features and target variable
X = data.drop(columns=['Petrol_Consumption'])  # Features (independent variables)
y = data['Petrol_Consumption']                 # Target variable (gas consumption)

# Step 4: Fit linear regression model
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_st

lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

# Step 5: Make predictions
predicted_gas_consumption = lin_reg.predict(X_test)

# Step 6: Calculate coefficients and intercept
coefficients = lin_reg.coef_
intercept = lin_reg.intercept_

print("Coefficients:", coefficients)
print("Intercept:", intercept)

# Step 7: Calculate mean squared error
mse = mean_squared_error(y_test, predicted_gas_consumption)
print("Mean Squared Error:", mse)
```

```
Missing values:
 Petrol_tax                     0
Average_income                  0
Paved_Highways                  0
Population_Driver_licence(%)    0
Petrol_Consumption              0
dtype: int64
Coefficients: [-3.69937459e+01 -5.65355145e-02 -4.38217137e-03  1.34686930e+03]
Intercept: 361.4508790665323
Mean Squared Error: 4083.255871745375
```

225 Write a program to create a Model using linear regression to predict the gas consumption using the csv file provided named “FuelConsumptionCo2.csv”. Do the required process in the data before making a model. Find predicted values, co-efficients, intercept and mean squared error. (Wherever required remove null values, convert categorical data into numeric data) (Print Output wherever required)

In [5]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Step 1: Load the dataset
data = pd.read_csv("FuelConsumptionCo2.csv")

# Step 2: Data preprocessing
# Check for any missing values
missing_values = data.isnull().sum()
print("Missing values:\n", missing_values)
```

```
# Step 3: Convert categorical data into numeric data (if any)
# In this case, we can use one-hot encoding for categorical variables like MAKE,VE
data = pd.get_dummies(data, columns=['MAKE', 'VEHICLECLASS', 'TRANSMISSION', 'FUEL
data.drop("MODEL",axis=1,inplace=True)
# Step 4: Split the dataset into features and target variable
X = data.drop(columns=['CO2EMISSIONS'])  # Features (independent variables)
y = data['CO2EMISSIONS']                  # Target variable (gas consumption)

# Step 5: Fit linear regression model
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_st

lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

# Step 6: Calculate coefficients and intercept
coefficients = lin_reg.coef_
intercept = lin_reg.intercept_

print("Coefficients:", coefficients)
print("Intercept:", intercept)

# Step 7: Make predictions
predicted_gas_consumption = lin_reg.predict(X_test)

# Step 8: Calculate mean squared error
mse = mean_squared_error(y_test, predicted_gas_consumption)
print("Mean Squared Error:", mse)
print("R2 score",r2_score(y_test, predicted_gas_consumption))
```

```
Missing values:
 MODELYEAR                   0
MAKE                        0
MODEL                       0
VEHICLECLASS                0
ENGINESIZE                  0
CYLINDERS                   0
TRANSMISSION                0
FUELTYPE                    0
FUELCONSUMPTION_CITY        0
FUELCONSUMPTION_HWY         0
FUELCONSUMPTION_COMB        0
FUELCONSUMPTION_COMB_MPG    0
CO2EMISSIONS                0
dtype: int64
Coefficients: [ 5.85403763e-14  1.28860594e+00  8.71236362e-01 -5.52357859e+00
 -2.59992462e+00  2.55006130e+01 -1.85139571e+00  6.02766933e+00
  6.22161203e+00  1.44944155e+01  1.01616819e+00  4.12753464e+00
  3.04605626e+00  2.86340169e+00  5.66734486e+00  3.34134309e+00
  4.21089091e+00  6.48478520e+00  2.12183911e+00  3.23227302e+00
  3.08688491e+00  2.31678581e+00  5.43291268e+00  2.47938597e+00
  2.53366170e+00  2.00465394e+01  5.35995802e+00  2.93331805e+00
  2.73832282e+00  1.11733272e+01  3.79271158e+00  3.62179561e+00
  2.03087706e+00  4.85372975e+00  1.01741890e+00 -9.11657749e-01
  5.84195570e+00  9.54433155e+00  3.54349860e+00  7.27167091e+00
  9.31506909e+00  1.19166305e+00  3.71741927e+00  1.58976157e+00
  1.85207959e+00 -2.40640155e+00 -1.12988434e+00 -1.88524045e+00
 -2.78811982e+00 -1.25375570e+00 -9.66943021e-01 -3.88222415e+00
 -7.81273958e-01 -1.77306882e+00 -3.41326540e+00 -2.37223741e+00
  3.06792089e-01 -5.99573888e-01  3.10109894e+00  5.00784287e+00
  7.18878269e+00  4.76393097e+00  2.96492894e+00  5.46380721e+00
  4.49557146e+00  7.27167091e+00  5.81705855e+00  7.48951004e+00
  3.30905762e+00  4.22467784e+00  4.18634633e+00  3.79917732e+00
  1.79497769e+00  2.34676513e+00  7.83453572e+00  7.09540931e-01
  6.45748565e+00 -2.05397999e+00  5.23368924e+00  5.18814275e+00
  4.86392792e+00 -1.43447822e+02 -3.42691685e+01 -3.40691790e+01]
Intercept: 134.41284638373273
Mean Squared Error: 33.66361575753647
R2 score 0.9918587522426805
```

226 "For the given RealEstate csv, write a python program satisfying following tasks to demonstrate application of machine learning through multiple linear regression as follows –

Given: -

Dataset RealEstate.csv

ML Library to be used scikit-learn

Dependent variable 'Y house price of unit area'

Independent variables 'X1 transaction date', 'X2 house age', 'X3 distance to the nearest MRT station', 'X4 number of

convenience stores', 'X5 latitude' and 'X6 longitude'

1. Import required libraries.
2. Load RealEstate dataset, create a dataframe and check datatypes of its attributes using appropriate method.
3. Remove 'No' column from the dataframe.
4. Check for any null values in features using appropriate method.
5. Create feature variables x and y as given above.
6. Create training and testing sets of feature variables with 70% of data for training and with random state of 110.
7. Create and fit regression model using appropriate method.
8. Use testing set created in step 6 to find and print the prediction of the outcome.
9. Find and print coefficient and mean squared error of the regression model."

In [6]:
```
# 1. Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,r2_score

# 2. Load RealEstate dataset, create a dataframe and check datatypes of its attrib
data = pd.read_csv("RealEstate.csv")
print("Data types of attributes:")
print(data.dtypes)

# 3. Remove 'No' column from the dataframe
data.drop(columns=['No'], inplace=True)

# 4. Check for any null values in features
null_values = data.isnull().sum()
print("Null values in features:\n", null_values)

# 5. Create feature variables x and y
X = data[['X1 transaction date', 'X2 house age', 'X3 distance to the nearest MRT s
          'X4 number of convenience stores', 'X5 latitude', 'X6 longitude']]
y = data['Y house price of unit area']

# 6. Create training and testing sets of feature variables with 70% of data for tr
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_st

# 7. Create and fit regression model
regression_model = LinearRegression()
regression_model.fit(X_train, y_train)

# 8. Use testing set created in step 6 to find and print the prediction of the out
predicted_values = regression_model.predict(X_test)
print("Predicted values:", predicted_values)

# 9. Find and print coefficient and mean squared error of the regression model
coefficients = regression_model.coef_
mse = mean_squared_error(y_test, predicted_values)
print("Coefficients:", coefficients)
```

```
print("Mean Squared Error:", mse)
print("R2 score",r2_score(y_test,predicted_values))
```

```
Data types of attributes:
No                                        int64
X1 transaction date                     float64
X2 house age                            float64
X3 distance to the nearest MRT station  float64
X4 number of convenience stores           int64
X5 latitude                             float64
X6 longitude                            float64
Y house price of unit area              float64
dtype: object
Null values in features:
 X1 transaction date                      0
X2 house age                             0
X3 distance to the nearest MRT station   0
X4 number of convenience stores          0
X5 latitude                              0
X6 longitude                             0
Y house price of unit area               0
dtype: int64
Predicted values: [40.06913335 32.56470479 29.80668902 44.60809493 39.96541473 44.3
4984726
 37.64834639 51.18801909 37.08044432 38.64449437 36.45757424 31.64776451
 43.03392987 16.64984039 36.61381207 31.21699197 39.55333345 41.89770923
 46.41953538 42.69067202 40.49560609 53.3447345  40.74436408 36.67049204
 46.86499408 42.78349352 47.25115307 40.53653503 42.57499139 50.88772177
 44.81813044 41.05114011 35.39386877 42.95199941 13.97578058 30.00463214
  0.32863779 39.56884707 47.9626967  34.0147913  40.23322474 49.74492584
 36.73990519 35.39784888 29.6071314  45.83432319 42.44745248 37.88237609
 31.55675944 47.517238   35.03089266 45.17666152 45.98711652 47.32685806
 54.18178829 15.18888047 43.78264851 44.75385635 37.69057227 39.34215811
 44.21812376 44.27270068 47.84570949 38.42626454 41.0626191  19.64522484
 47.9626967  43.85961508 37.99756745 43.80152052 42.39390912 42.57768877
 44.73329588 32.31758114 46.89135756 38.92015239 40.90438603 43.95669962
 34.28075691 25.5851042  16.26403643 45.66098257 42.05300968 10.81561309
 31.65043887 49.10789869 47.40016786 13.5604683  46.02203009 44.82145569
 34.76747867 35.40469328 31.48202917 45.59192922 42.0246353  42.54274755
 14.70678074 43.68554348 27.69149444 43.0460661  27.63908623 35.23545719
 45.89665228 50.31290226 42.86214486 44.61671482 49.59887911 32.11577476
 31.86965512 38.39003281 44.58865137 25.50284004 46.20113858 37.45131001
 14.70678074 44.2357441   9.25548578 37.23816575 38.97851394 47.40016786
 39.50115693 32.15412829 46.99435489 47.07907957 53.12308545]
Coefficients: [ 6.84308941e+00 -2.25466590e-01 -5.18046265e-03  1.12369103e+00
  1.87739728e+02 -4.29300312e+01]
Mean Squared Error: 77.1480259825389
R2 score 0.5447865064578654
```

227 Write a program to create a Model using linear regression to predict the charges of insurance using the csv file provided named “insurance.csv”. Do the required process in the data before making a model. Find predicted values, co-efficients, intercept and mean squared error.

In [7]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

```
# Load the data
data = pd.read_csv('insurance.csv')

# Display the first few rows of the dataset
print(data.head())

# Preprocess the data
# Convert categorical variables into dummy/indicator variables
data = pd.get_dummies(data, drop_first=True)

# Define the features and the target variable
X = data.drop('expenses', axis=1)
y = data['expenses']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_st

# Create and train the linear regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)

# Find the coefficients and intercept
coefficients = model.coef_
intercept = model.intercept_

# Calculate the mean squared error
mse = mean_squared_error(y_test, y_pred)

# Display the results
print("Coefficients:", coefficients)
print("Intercept:", intercept)
print("Mean Squared Error:", mse)
print("Predicted Values:", y_pred[:10])
print("r2_score",r2_score(y_test,y_pred))
```

```
   age     sex   bmi  children smoker     region  expenses
0   19  female  27.9         0    yes  southwest  16884.92
1   18    male  33.8         1     no  southeast   1725.55
2   28    male  33.0         3     no  southeast   4449.46
3   33    male  22.7         0     no  northwest  21984.47
4   32    male  28.9         0     no  northwest   3866.86
Coefficients: [  261.28251281   348.96600937   424.41067944   104.99524716
 23627.8945956   -486.46995207  -970.61815579  -925.06307896]
Intercept: -12376.785237284646
Mean Squared Error: 33777093.10084606
Predicted Values: [ 9023.69263444  7011.89555317 36873.90587849  9502.39412647
 26966.01809591 11081.80484819   -47.15223497 17189.63263317
   976.23708368 11333.13969304]
r2_score 0.7696351080608885
```

228 "Write a program to create a Model using linear regression to predict the wine quality using the csv file provided named “winequality.csv”. Do the required process in the data before making a model.

If you find any null value in “winequality.csv” then replace null value with mean value of respected columns. Find co-efficient, intercept and mean squared error.

also Predict the quality of red wine for the following data:
fixed acidity: 8

volatile acidity: 0.4

citric acid: 0.40

residual sugar: 15

chlorides: 0.048

free sulfur dioxide: 40

total sulfur dioxide: 150

density: 0.99

pH: 3

sulphates: 0.45

alcohol: 10.5"

In [8]:
```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Step 1: Load the dataset
data = pd.read_csv("winequality.csv")

# Step 2: Replace null values with mean value of respective columns
data.fillna(data.mean(), inplace=True)

# Step 3: Split the dataset into features and target variable
X = data.drop(columns=['quality',"Id"])  # Features (independent variables)
y = data['quality']                # Target variable

# Step 4: Create training and testing sets of feature variables with 70% of data f
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_st

# Step 5: Create and fit linear regression model
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

# Step 6: Calculate coefficients and intercept
coefficients = lin_reg.coef_
intercept = lin_reg.intercept_

print("Coefficients:", coefficients)
print("Intercept:", intercept)

# Step 7: Calculate mean squared error
predicted_quality = lin_reg.predict(X_test)
mse = mean_squared_error(y_test, predicted_quality)
print("Mean Squared Error:", mse)

# Step 8: Predict the quality of red wine for the provided data
new_data = pd.DataFrame({
```

```
    'fixed acidity': [8],
    'volatile acidity': [0.4],
    'citric acid': [0.40],
    'residual sugar': [15],
    'chlorides': [0.048],
    'free sulfur dioxide': [40],
    'total sulfur dioxide': [150],
    'density': [0.99],
    'pH': [3],
    'sulphates': [0.45],
    'alcohol': [10.5]
})

predicted_red_wine_quality = lin_reg.predict(new_data)
print("Predicted quality of red wine:", predicted_red_wine_quality)
```

```
Coefficients: [ 4.42109900e-02 -1.34670790e+00 -3.14932885e-01 -2.68489475e-03
 -1.87846120e+00  2.37296538e-03 -2.77800524e-03 -1.60760914e+01
 -2.85153430e-01  9.58575428e-01  2.74942710e-01]
Intercept: 19.808325962059342
Mean Squared Error: 0.3855917837384169
Predicted quality of red wine: [5.59260242]
```

229 "Consider variables x and y created from a pandas dataframe “car.csv” . Create new column named “Age_car” (Age_car=2023-year) For multiple linear regression problem, x contains the independent variables (Age_car , Driven_kms , Fuel_Type , Selling_type , Transmission) and y contains the dependent (Selling_Price) variable which is to be predicted.Write a Python program to spilt x and y into training and testing datasets with a 20% split. Then create a multiple linear regression model using the training data and print its coefficients ,intercept and mean squared error."

In [9]:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Load the data
data = pd.read_csv('car.csv')
print(data.head())

# Create a new column 'Age_car'
data['Age_car'] = 2023 - data['Year']

# Select independent variables and the dependent variable
X = data[['Age_car', 'Kms_Driven', 'Fuel_Type', 'Seller_Type', 'Transmission']]
y = data['Selling_Price']

# Convert categorical variables into dummy/indicator variables
X = pd.get_dummies(X, drop_first=True)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_st

# Create and train the multiple linear regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)
```

```
# Find the coefficients and intercept
coefficients = model.coef_
intercept = model.intercept_

# Calculate the mean squared error
mse = mean_squared_error(y_test, y_pred)

# Display the results
print("Coefficients:", coefficients)
print("Intercept:", intercept)
print("Mean Squared Error:", mse)
```

```
  Car_Name  Year  Selling_Price  Present_Price  Kms_Driven Fuel_Type  \
0     ritz  2014           3.35           5.59       27000    Petrol
1      sx4  2013           4.75           9.54       43000    Diesel
2     ciaz  2017           7.25           9.85        6900    Petrol
3  wagon r  2011           2.85           4.15        5200    Petrol
4    swift  2014           4.60           6.87       42450    Diesel

  Seller_Type Transmission  Owner
0      Dealer       Manual      0
1      Dealer       Manual      0
2      Dealer       Manual      0
3      Dealer       Manual      0
4      Dealer       Manual      0
Coefficients: [-2.79387747e-01 -4.48042783e-06  6.40927579e+00  1.24649346e+00
 -4.11102502e+00 -4.80303386e+00]
Intercept: 10.888445135748384
Mean Squared Error: 9.418108720809974
```

230 "Write a python program for following tasks -

Make sure all the required libraries are imported at the beginning of the Jupyter Notebook.
a) Load the windpower dataset into the notebook.

b) Check records for any missing values and remove them permanently using appropriate method.

c) Treat windspeed as the independent variable and power as the dependent variable. Split the dataset into training (75%) and testing (25%) sets using random state of 42.

d) Use scikit-learn to train polynomial regression models of degrees 3, 4, 5, and 6.

e) Predict power values on the test set and print the Mean Squared Error (MSE) and R^2 score (coefficient of determination) for each polynomial degree.

f) Plot actual vs. predicted power values for all polynomial degrees on the same graph for comparison and label the plot accordingly.

g) Create a plot showing MSE vs. polynomial degree. Which degree has the lowest MSE? Write in the comment.

h) Print the coefficients and intercept for each trained polynomial regression model.

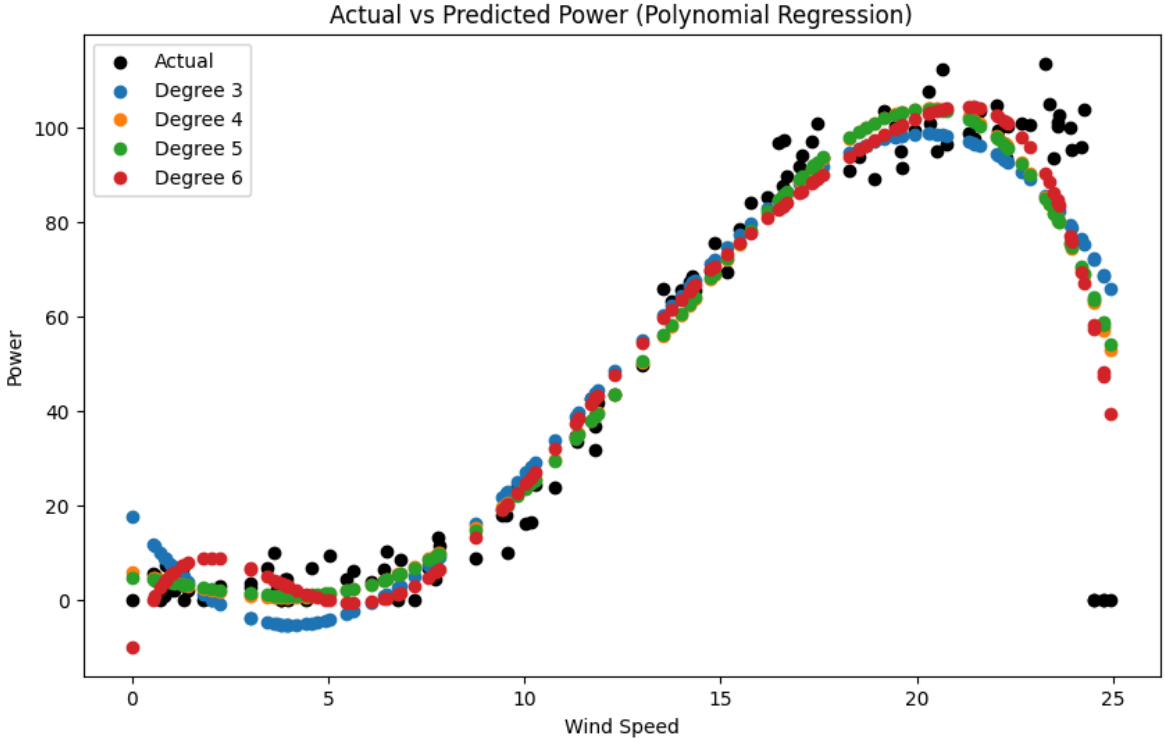
i) Use each model to predict power output for wind speeds of 5, 10, 15, and 20 m/s, and print the predictions."

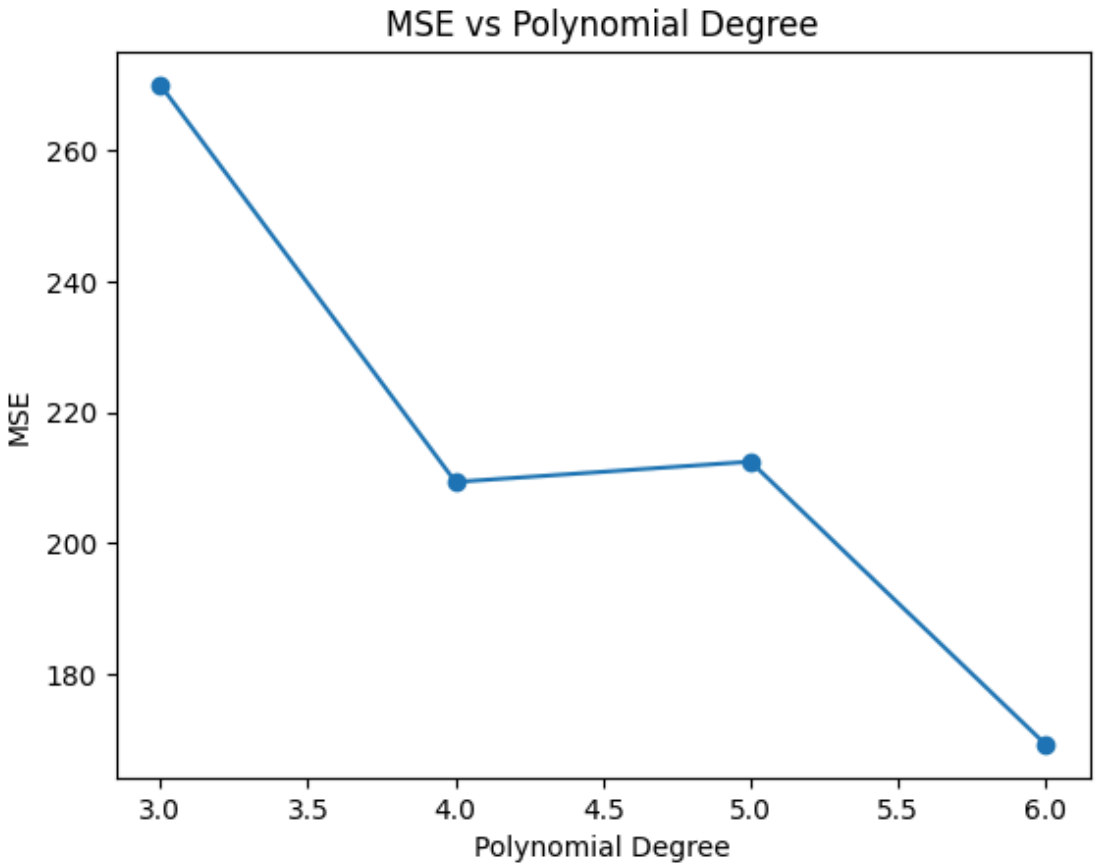
In [12]:
```
# ================================
# Import Required Libraries
# ================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# ================================
# (a) Load Dataset
# ================================
data = pd.read_csv("windpower.csv")

print(data.head())

# ================================
# (b) Handle Missing Values
# ================================
print("Missing Values:\n", data.isnull().sum())

# Remove missing values permanently
data = data.dropna()

# ================================
# (c) Define X and y + Split
# ================================
X = data[['Wind speed (m/s)']]   # Independent
y = data['Power (kW)']          # Dependent

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# ================================
# (d) Train Polynomial Models
# ================================
degrees = [3, 4, 5, 6]

models = {}
predictions = {}
mse_list = []
r2_list = []

# ================================
# (e) Train, Predict, Evaluate
# ================================
for d in degrees:
    poly = PolynomialFeatures(degree=d)

    X_train_poly = poly.fit_transform(X_train)
    X_test_poly = poly.transform(X_test)

    model = LinearRegression()
```

```
    model.fit(X_train_poly, y_train)

    y_pred = model.predict(X_test_poly)

    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    models[d] = (model, poly)
    predictions[d] = y_pred
    mse_list.append(mse)
    r2_list.append(r2)

    print(f"\nDegree {d}:")
    print("MSE:", mse)
    print("R2 Score:", r2)

# ================================
# (f) Plot Actual vs Predicted
# ================================
plt.figure(figsize=(10,6))

plt.scatter(X_test, y_test, color='black', label='Actual')

for d in degrees:
    plt.scatter(X_test, predictions[d], label=f'Degree {d}')

plt.xlabel("Wind Speed")
plt.ylabel("Power")
plt.title("Actual vs Predicted Power (Polynomial Regression)")
plt.legend()
plt.show()

# ================================
# (g) Plot MSE vs Degree
# ================================
plt.figure()

plt.plot(degrees, mse_list, marker='o')
plt.xlabel("Polynomial Degree")
plt.ylabel("MSE")
plt.title("MSE vs Polynomial Degree")
plt.show()

# Find best degree
best_degree = degrees[np.argmin(mse_list)]
print("\nBest Degree (Lowest MSE):", best_degree)

# ================================
# (h) Print Coefficients
# ================================
for d in degrees:
    model, poly = models[d]
    print(f"\nDegree {d} Coefficients:")
    print("Intercept:", model.intercept_)
    print("Coefficients:", model.coef_)

# ================================
# (i) Predict for New Values
# ================================
new_wind = np.array([[5], [10], [15], [20]])
```

```
for d in degrees:
    model, poly = models[d]
    new_poly = poly.transform(new_wind)
    pred = model.predict(new_poly)

    print(f"\nPredictions for Degree {d}:")
    for ws, p in zip([5,10,15,20], pred):
        print(f"Wind Speed {ws} → Power = {p}")
```

```
   Power (kW)  Wind speed (m/s)
0         0.0             0.000
1         0.0             0.125
2         0.0             0.150
3         0.0             0.225
4         0.0             0.275
Missing Values:
 Power (kW)          3
Wind speed (m/s)    5
dtype: int64

Degree 3:
MSE: 270.0250095300159
R2 Score: 0.8539264141973721

Degree 4:
MSE: 209.31845107836853
R2 Score: 0.8867664266473421

Degree 5:
MSE: 212.42288613451007
R2 Score: 0.8850870416106328

Degree 6:
MSE: 169.17010460788921
R2 Score: 0.9084852035235902
```





```
Best Degree (Lowest MSE): 6

Degree 3 Coefficients:
Intercept: 17.775342331044335
Coefficients: [  0.          -12.17377575   1.8029668   -0.04960797]

Degree 4 Coefficients:
Intercept: 5.863727006185634
Coefficients: [ 0.          -2.10494179 -0.036268    0.06568915 -0.00231786]

Degree 5 Coefficients:
Intercept: 4.739942925620888
Coefficients: [ 0.00000000e+00 -6.91240269e-01 -4.31617906e-01  1.07790122e-01
 -4.21343552e-03  3.03911610e-05]

Degree 6 Coefficients:
Intercept: -9.916620514063588
Coefficients: [ 0.00000000e+00  2.39063436e+01 -1.00641730e+01  1.62983192e+00
 -1.17629556e-01  4.01186376e-03 -5.30687163e-05]

Predictions for Degree 3:
Wind Speed 5 → Power = -4.220362224742679
Wind Speed 10 → Power = 26.726298078704623
Wind Speed 15 → Power = 73.40934798339262
Wind Speed 20 → Power = 98.62281223132766

Predictions for Degree 4:
Wind Speed 5 → Power = 1.194801843474668
Wind Speed 10 → Power = 23.69809950560926
Wind Speed 15 → Power = 70.48872280095827
Wind Speed 20 → Power = 103.91393443390194

Predictions for Degree 5:
Wind Speed 5 → Power = 1.4286343581236487
Wind Speed 10 → Power = 23.360632572194326
Wind Speed 15 → Power = 70.82208666107363
Wind Speed 20 → Power = 103.69098368309949

Predictions for Degree 6:
Wind Speed 5 → Power = -0.07083468861747377
Wind Speed 10 → Power = 24.383531791979358
Wind Speed 15 → Power = 71.94924385011024
Wind Speed 20 → Power = 102.03357900436191
```

```
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X does not have valid feature names, but PolynomialFeatures was fitted with feature names
  warnings.warn(
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X does not have valid feature names, but PolynomialFeatures was fitted with feature names
  warnings.warn(
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X does not have valid feature names, but PolynomialFeatures was fitted with feature names
  warnings.warn(
C:\Users\VISHAL\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X does not have valid feature names, but PolynomialFeatures was fitted with feature names
  warnings.warn(
```

In [ ]: